

Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming

Paul Biberstein

paulbib@seas.upenn.edu
University of Pennsylvania
Philadelphia, Pennsylvania, USA

Joseph Devietti

devietti@seas.upenn.edu
University of Pennsylvania
Philadelphia, Pennsylvania, USA

Ziyang Li

liby99@seas.upenn.edu
Johns Hopkins University
Baltimore, Maryland, USA

Mayur Naik

mhnaik@seas.upenn.edu
University of Pennsylvania
Philadelphia, Pennsylvania, USA

Abstract

Neurosymbolic programs combine deep learning with symbolic reasoning to achieve better data efficiency, interpretability, and generalizability compared to standalone deep learning approaches. However, existing neurosymbolic learning frameworks implement an uneasy marriage between a highly scalable, GPU-accelerated neural component and a slower symbolic component that runs on CPUs.

We propose Lobster, a unified framework for harnessing GPUs in an end-to-end manner for neurosymbolic learning. Lobster maps a general neurosymbolic language based on Datalog to the GPU programming paradigm. This mapping is implemented via compilation to a new intermediate language called APM. The extra abstraction provided by apm allows Lobster to be both flexible, supporting discrete, probabilistic, and differentiable modes of reasoning on GPU hardware with a library of provenance semirings, and performant, implementing new optimization passes.

We demonstrate that Lobster programs can solve interesting problems spanning the domains of natural language processing, image processing, program reasoning, bioinformatics, and planning. On a suite of 9 applications, Lobster achieves an average speedup of 3.9x over Scallop, a state-of-the-art neurosymbolic framework, and enables scaling of neurosymbolic solutions to previously infeasible tasks.

CCS Concepts: • **Theory of computation** → **Probabilistic computation**; • **Computer systems organization** → **Parallel architectures**; • **Software and its engineering** → **Compilers**.

Keywords: neurosymbolic programming, GPU acceleration, Datalog, compiler optimizations



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2165-6/26/03

<https://doi.org/10.1145/3760250.3762232>

ACM Reference Format:

Paul Biberstein, Ziyang Li, Joseph Devietti, and Mayur Naik. 2026. Lobster: A GPU-Accelerated Framework for Neurosymbolic Programming. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3760250.3762232>

1 Introduction

Deep learning and classical algorithms represent two predominant paradigms of modern programming. Classical algorithms excel at problems with clearly defined rules and structured data, such as sorting a list of numbers or finding a shortest path in a graph. In contrast, deep learning is well suited to contexts where classical algorithmic approaches become intractable, particularly for problems involving noisy, complex, and high-dimensional data—such as detecting objects in an image or parsing natural language text.

Many machine learning problems in different domains demand the complementary capabilities of these two paradigms. Neurosymbolic programming [6] is an emerging approach to solve such problems by suitably decomposing the computation between a neural network and a symbolic program. The resulting *neurosymbolic programs* have been demonstrated to achieve better data efficiency, interpretability, and generalizability compared to standalone deep learning approaches. Such properties are crucial for various safety-critical domains such as system security [25], cyber-physical systems [54], and healthcare [48].

Recent frameworks such as DeepProbLog [28], Scallop [26], and ISED [41] have enhanced the programmability and accessibility of neurosymbolic applications. Figure 1 illustrates a neurosymbolic program for solving a binary image-classification problem [45]. The symbolic program is specified in Datalog [2], a declarative language. Crucially, by using a differentiable Datalog engine, gradients can be back-propagated through the program to train the neural network, thereby enabling automatic learning of relevant image features without manual engineering.

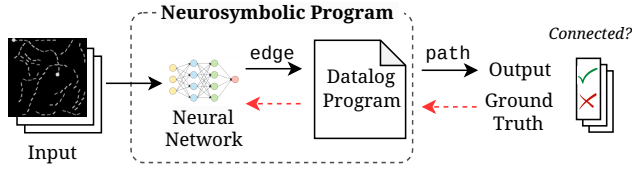


Figure 1. An example neurosymbolic program.

Despite their benefits, neurosymbolic programs using these frameworks incur significant computational overhead during training and inference. The scalability challenges stem primarily from managing probabilistic data and maintaining end-to-end differentiability. For instance, in the neurosymbolic program in Figure 1, a neural network identifies edges in an image. These edges are represented in an edge relation, where each tuple has an associated probability representing the network’s confidence. The symbolic program computes the transitive closure to produce a path relation, where the probability of each path tuple must take into account all the possible ways to derive it from the edge tuples, and their associated probabilities. Since it is often intractable to perform exact probabilistic reasoning, approximated probabilistic inference is employed, though this has only limited scalability benefits. Differentiability further complicates the problem by requiring us to track each input’s contribution to the output, increasing space and time complexity due to the extra book-keeping required for gradients.

In this paper, we propose Lobster, a GPU-accelerated framework designed to enhance the scalability of neurosymbolic programming. Lobster’s core innovation is efficiently mapping Datalog—a logic programming language shown to be effective in neurosymbolic contexts [26]—onto GPU architectures, for different modes of reasoning: discrete, probabilistic, and differentiable. The key architectural choice that enables the efficiency of this mapping is compilation to a new intermediate language, called APM, which is expressive enough to support complex reasoning but at the same time restricted enough to ensure massively parallel execution.

Supporting both advanced reasoning modes and GPU acceleration makes Lobster the first system of its kind, as shown in Figure 2. While various engines exist for discrete [36], probabilistic [13], and differentiable [26] settings, they are limited to CPU runtimes with single- or multi-threading. Other work [38, 44] implements GPU-accelerated Datalog execution, but does not support differentiable and probabilistic reasoning. In contrast, Lobster handles general neurosymbolic queries with multiple reasoning modes within a unified, GPU-accelerated framework of *provenance semirings* [17]. This requires fundamental changes to the program semantics compared to discrete Datalog, enriching the runtime with semiring-based tags that propagate alongside data throughout the computation. To support this efficiently on

modern hardware, Lobster introduces GPU-optimized operators specifically tailored for tagged computation. By supporting a library of 7 common semirings in the literature, Lobster allows employing reasoning in a particular mode (e.g. probabilistic) by simply selecting a suitable corresponding semiring (e.g. Top- k -Proofs).

In addition to describing the compilation to and execution of APM, we propose a number of optimizations unique to Lobster and discuss considerations for our implementation—a fully-fledged compiler and runtime written in Rust—that can execute existing Datalog-based neurosymbolic programs on GPUs without any modifications.

In summary, the core contributions of this paper are:

- We introduce Lobster, the first GPU-accelerated neurosymbolic programming framework.
- We propose the APM language and show how to compile Datalog to APM.
- We implement a compiler and runtime for Lobster using Rust and CUDA.
- We evaluate Lobster on an extensive set of discrete, probabilistic, and differentiable benchmarks, showing that Lobster consistently outperforms prior systems, including more specialized ones. Lobster achieves a speedup of 3.9x on average and upto $>100\times$ over Scallop [27], the closest existing state-of-the-art system.

The code for Lobster is publicly available at <https://github.com/P-bibs/Lobster>.

The rest of the paper is organized as follows. We first give an illustrative overview of Lobster in Section 2. Then we introduce Lobster’s core language and compiler (Section 3), optimizations (Section 4), and implementation details (Section 5). Finally, we present experimental results (Section 6) and related work (Section 7).

2 Illustrative Overview

We illustrate Lobster using an example image-reasoning task Pathfinder [45] wherein the goal is to determine whether two dots in the input image are connected by dashed lines (Figure 3). Neurosymbolic methods have been shown to achieve greater accuracy than purely neural methods on this task [27], but the symbolic performance bottleneck quickly appears as the lengths and complexities of lines (and therefore reasoning chain size) increase.

2.1 A Neurosymbolic Solution

As with many visual reasoning tasks, the Pathfinder task presents an obvious opportunity for neural/symbolic decomposition: a neural network can extract a structured representation from the image, and a symbolic engine can be used to reason over this representation.

More formally, we choose a discretization factor n and overlay the lattice graph $G_{n,n}$ on the image, where each vertex corresponds to a square region of pixels. A convolutional

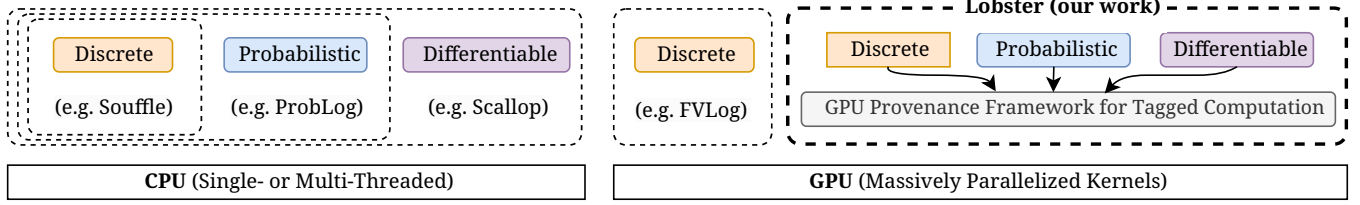
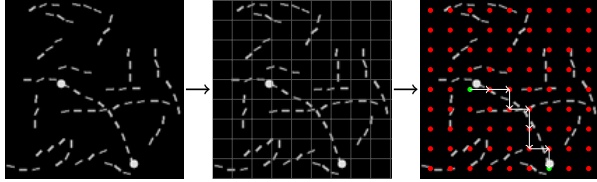


Figure 2. High-level comparison between Lobster and existing frameworks.



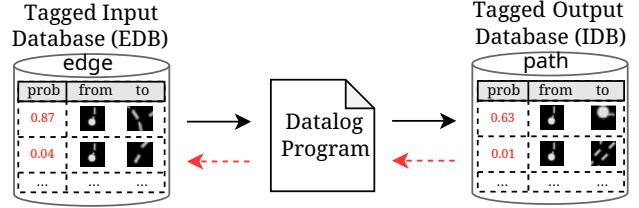
(a) An example graph a trained model extracts for the Pathfinder task. Green vertices are endpoints and white edges are predicted connectivity. Additional edges not relevant to endpoint connectivity are elided for clarity.

```

1  type Cell = u32
2  type edge(x: Cell, y: Cell)
3  type is_endpoint(x: Cell)
4
5  rel path(x, y) :-
6    edge(x, y) or (path(x, z) and edge(z, y)).
7  rel endpoints_connected() :- is_endpoint(x),
8    is_endpoint(y), path(x, y), x != y.

```

(c) A Lobster program that computes reachability information for a graph extracted by a neural network.



(b) The input and output for the Pathfinder Datalog program.

Accuracy	Neural	Neurosymbolic
Pathfinder	71.40	87.42
Pathfinder-x	49.80	89.46

(d) Accuracy of neural methods vs. neurosymbolic methods [27]. “Pathfinder-x” is a more challenging dataset.

	Scallop	Lobster
Training Time	41 hr.	32 hr

(e) Training time of Scallop versus Lobster.

Figure 3. The overall pipeline, symbolic program, and the acceleration result of using Lobster for the Pathfinder task.

neural network model [22] predicts probabilities for each edge in $G_{n,n}$, with high probability indicating confidence that two grid cells are connected by a dashed line. Additionally, the model predicts a probability corresponding to the presence of a dot in each grid cell. The symbolic component then computes the transitive closure of the graph and queries the result to determine the probability two dots are connected.

2.2 A Scallop Implementation

We present the high-level pipeline of implementing a neurosymbolic solution using Scallop, a state-of-the-art neurosymbolic framework, in Figure 3a. We specify the symbolic component as a Datalog program which is shown in Figure 3c. Datalog offers an intuitive interface for defining data types (line 1) and relation types (lines 2-3) which we use to specify the inputs. Notice that the graph is encoded as a binary relation between Cells, which we represent with unsigned integers. Our program contains recursive rules, which are key for modeling reachability in a graph. The actual reachability rule is defined succinctly in line 6.

Notably, the probabilistic reasoning semantics is abstracted away. The edge relation in Figure 3c (line 2) contains probabilistic facts extracted by the underlying neural network. For example, a fact $0.97 :: \text{edge}(0, 1)$ represents a prediction that a dotted line connects cell 0 and cell 1 with the probability of 0.97. All the derived facts, such as $\text{path}(i, j)$, will carry probabilities computed from a concrete set of edges that are used to derive them. A proof for $\text{path}(1, 5)$, for example, could be the conjunction $\text{is_endpoint}(1) \wedge \text{edge}(1, 2) \wedge \dots \wedge \text{edge}(4, 5) \wedge \text{is_endpoint}(5)$, representing a concrete path formed by a subset of the probabilistic input facts. This computation is manifested by the underlying provenance framework within Lobster. In our solution, we use a probabilistic provenance called *diff-top-1-proofs* which instruments the program to carry the most-likely path between any two cells.

Since the symbolic engine is differentiable, the entire neurosymbolic system can be trained end-to-end, resulting in a trained neural network despite only having “yes”/“no” supervision for each image. The neurosymbolic solution achieves